

MANUAL DE EJERCICIOS PATRONES DE SOFTWARE

***ELABORADO POR:
LEYDI JOHANA POLO AMADOR***

***GRUPO DE INVESTIGACIÓN:
GRIIS***

***UNIDADES TECNOLÓGICAS DE SANTANDER
FACULTAD DE CIENCIAS NATURALES E INGENIERÍAS
PROGRAMA INGENIERÍA DE SISTEMAS
BUCARAMANGA
2020***

PATRONES DE SOFTWARE



Los Patrones de Diseño de software son soluciones para problemas típicos y recurrentes que nos podemos encontrar a la hora de desarrollar una aplicación orientada a objetos. Aunque la aplicación sea única, tendrá cosas comunes con otras aplicaciones ya desarrolladas: Acceso a datos, creación de clases y objetos, operaciones entre diferentes sistemas.

Cuando se desarrolla una aplicación software es frecuente encontrarse en la situación de tener que volver a resolver problemas similares a otros que ya hemos solucionado anteriormente, y debemos volver a hacerlo partiendo de cero una y otra y otra vez (incluso dentro del mismo proyecto).

Debido a ello y basándose en la programación orientada a objetos surgieron los patrones de diseño, donde cada uno de ellos define la solución para resolver un determinado problema, facilitando además la reutilización del código fuente.

Dependiendo de su finalidad pueden ser:

- **Patrones de creación:** utilizados para crear y configurar de clases y objetos.
- **Patrones estructurales:** su objetivo es desacoplar las interfaces e implementar clases y objetos. Crean grupos de objetos.
- **Patrones de comportamiento:** se centran en la interacción entre asociaciones de clases y objetos definiendo cómo se comunican entre sí.

Los patrones de diseño son muy útiles por muchos motivos: Ahorran tiempo; te ayudan a estar seguro de la validez de su código; establecen un lenguaje común entre todos los miembros del equipo.

CATEGORÍAS DE LOS PATRONES

	CREACIÓN	ESTRUCTURALES	COMPORTAMIENTO
CLASES	Factory Method		Interpreter Template Method
OBJETOS	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Flyweight Facade Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

PATRONES DE CREACIÓN

Son los que facilitan la tarea de creación de nuevos objetos, de tal forma que el proceso de creación pueda ser desacoplado de la implementación del resto del sistema.

- **Creacional de la Clase**

Los patrones creacionales de Clases usan la herencia como un mecanismo para lograr la instanciación de la Clase. Por ejemplo el Factory Method.

- **Creacional del Objeto**

Los patrones creacionales de objetos son más escalables y dinámicos comparados de los patrones creacionales de Clases. Por ejemplo la Abstract Factory y el patrón Singleton.

ABSTRACT FACTORY

El objetivo de este patrón de diseño es la creación de objetos agrupados en familias; sin tener que conocer las clases concretas destinadas a la creación de los objetos.

DIAGRAMA DE CLASES

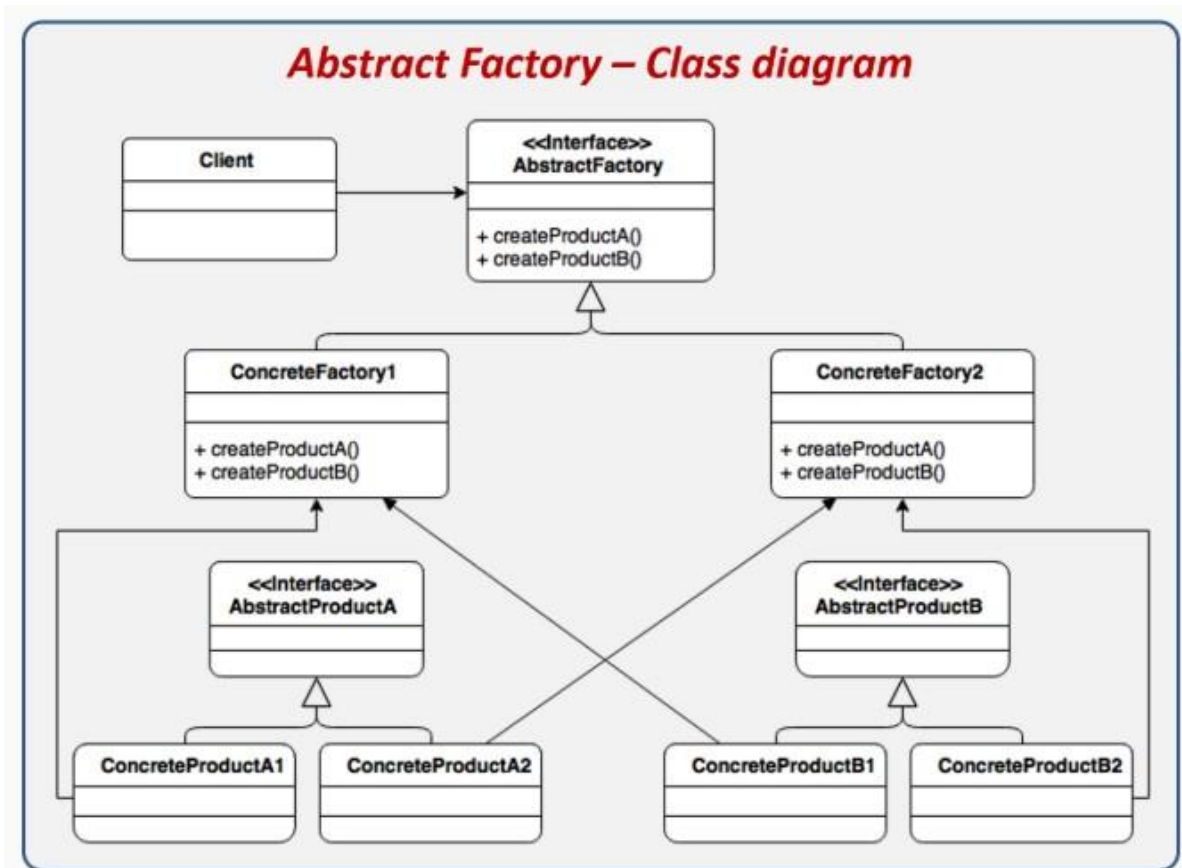


Diagrama de Clases Genérico

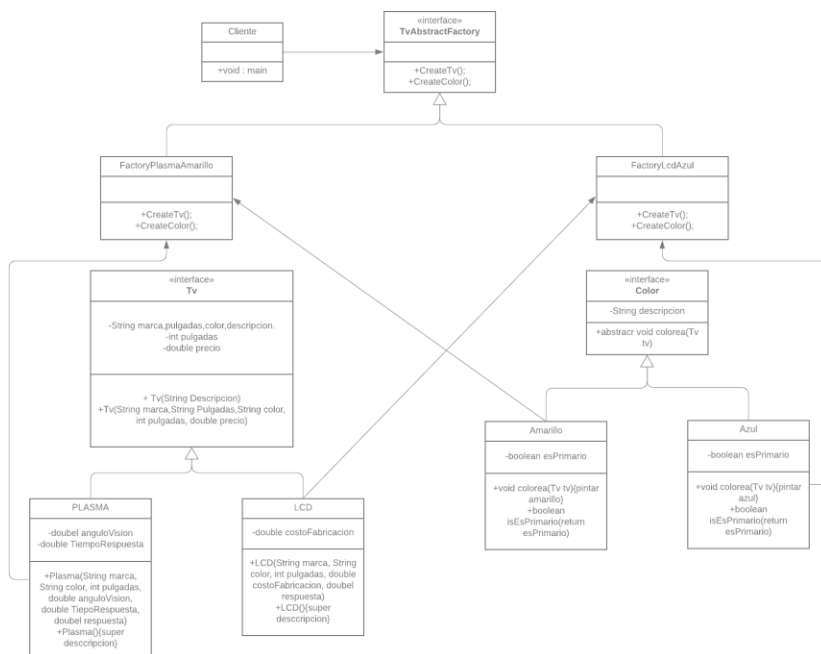
APLICACIÓN

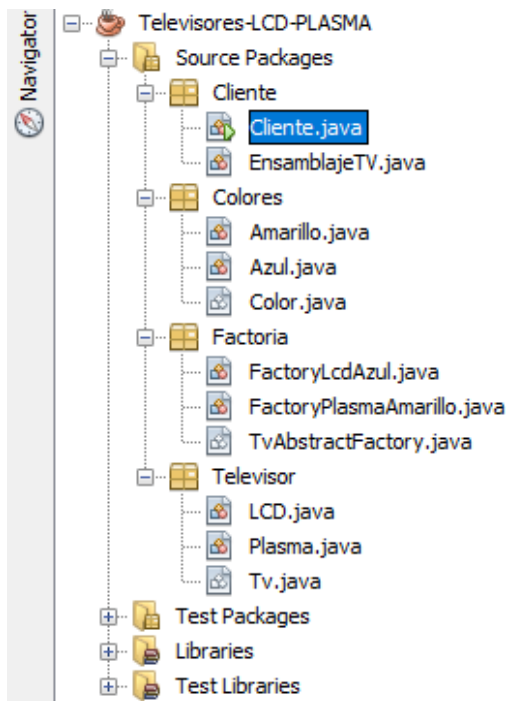
Hagamos de cuenta que tenemos dos familias de objetos:

1) La clase TV, que tiene dos hijas: Plasma y LCD.

2) La clase Color, que tiene dos hijas: Amarillo y Azul (¡los mejores colores, sin duda!).

Más allá de todos los atributos/métodos que puedan tener la clase Color y TV, lo importante aquí es destacar que Color define un método abstracto. Escenario: nuestra empresa se dedica a darle un formato estético específico a los televisores LCD y Plasma. Se ha decidido que todos los LCD que saldrán al mercado serán azules y los plasma serán amarillos. Ahora bien, una solución simple sería en la clase Azul colocar el LCD y en la clase Amarillo colocar el Plasma y todo funcionaría de maravillas. ¿Cuál sería el problema? Que esta todo hardcodedo. Esto quiere decir que el hecho de que los LCD sean azules y los plasmas amarillos es una decisión del negocio y, como tal, puede variar (y de hecho el negocio varía constantemente). Por ejemplo, que pasa si mañana se desea agrega otro color o me cambian el color del LCD o mucho peor, ¿qué pasa si se crea otro producto LED y también se lo quiere pintar de Azul?





Estructura de Carpetas

Clase Tv Abstract

```
package Televisor;

public abstract class Tv {
    private String marca;
    private int pulgadas;
    private String color;
    private String descripcion;
    private double precio;

    public Tv(String descripcion) {
        this.descripcion = descripcion;
    }

    public Tv(String marca, int pulgadas, String color, double precio) {
        this.marca = marca;
        this.pulgadas = pulgadas;
        this.color = color;
        this.precio = precio;
    }

    public String getMarca() {...3 lines }
    public void setMarca(String marca) {...3 lines }
    public int getPulgadas() {...3 lines }
    public void setPulgadas(int pulgadas) {...3 lines }
    public String getColor() {...3 lines }
    public void setColor(String color) {...3 lines }
    public String getDescripcion() {...3 lines }
    public void setDescripcion(String descripcion) {...3 lines }
    public double getPrecio() {...3 lines }
    public void setPrecio(double precio) {...3 lines }
}
```

Clase Color Abstract

```
package Colores;

import Televisor.Tv;

public abstract class Color {
    private String descripcion;
    public abstract void colorea(Tv tv);

    public String getDescripcion() {
        return descripcion;
    }

    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }
}
```

Clase Plasma

```
package Televisor;

public class Plasma extends Tv {
    private double anguloVision;
    private double tiempoRespuesta;
    public Plasma(double anguloVision, double tiempoRespuesta, String marca,
        int pulgadas, String color, double precio) {
        super(marca, pulgadas, color, precio);
        this.anguloVision = anguloVision;
        this.tiempoRespuesta = tiempoRespuesta;
    }

    public Plasma() {
        super("Plasma...Proximamente sera LED");
    }

    public double getAnguloVision() {...3 lines }
    public void setAnguloVision(double anguloVision) {...3 lines }
    public double getTiempoRespuesta() {...3 lines }
    public void setTiempoRespuesta(double tiempoRespuesta) {...3 lines }
}
```

Clase LCD

```
package Televisor;

public class LCD extends Tv{
    private double costoFabricacion;
    public LCD(double costoFabricacion, String marca, int pulgadas, String color, double precio) {
        super(marca, pulgadas, color, precio);
        this.costoFabricacion = costoFabricacion;
    }

    public LCD() {
        super("LCD");
    }

    public double getCostoFabricacion() {...3 lines }
    public void setCostoFabricacion(double costoFabricacion) {...3 lines }
}
```


Clase Azul

```
2 package Colores;
3
4 import Televisor.Tv;
5
6 public class Azul extends Color{
7     boolean esPrimario;
8     @Override
9     public void colorea(Tv tv) {
10         System.out.println("Pintando de azul el " + tv.getDescripcion());
11     }
12     public boolean isEsPrimario(){
13         return esPrimario;
14     }
15     public void setEsPrimario(boolean esPrimario) {
16         this.esPrimario = esPrimario;
17     }
18 }
```

Clase Amarillo

```
package Colores;

import Televisor.Tv;

public class Amarillo extends Color {
    boolean esPrimario;
    @Override
    public void colorea(Tv tv) {
        System.out.println("Pintando de amarillo el " + tv.getDescripcion());
    }
    public boolean isEsPrimario(){
        return esPrimario;
    }
    public void setEsPrimario(boolean esPrimario) {
        this.esPrimario = esPrimario;
    }
}
```

Para evitar un dolor de cabeza conviene separar estas familias y utilizar el Abstract Factory:
Clase TvAbstractFactory Abstract

```
package Factoria;

import Colores.Color;
import Televisor.Tv;

public abstract class TvAbstractFactory {
    public abstract Tv createTV();
    public abstract Color createColor();
}
```

Y los Factory Concretos, que relacionan las familias:
Clase FactoryPlasmaAmarillo

```
package Factoria;

import Colores.Amarillo;
import Colores.Color;
import Televisor.Plasma;
import Televisor.Tv;

public class FactoryPlasmaAmarillo extends TvAbstractFactory {
    @Override
    public Tv createTV() {
        return new Plasma();
    }
    @Override
    public Color createColor() {
        return new Amarillo();
    }
}
```

Clase FactoryLcdAzul

```
package Fectoria;
import Colores.Azul;
import Colores.Color;
import Televisor.LCD;
import Televisor.Tv;
public class FactoryLcdAzul extends TvAbstractFactory{
    @Override
    public Tv createTV() {
        return new LCD();
    }
    @Override
    public Color createColor() {
        return new Azul();
    }
}
```

Para ordenar un poco las cosas se va a crear un gestor de factorías

Clase EnsamblajeTV

```
package Cliente;

import Colores.Color;
import Televisor.Tv;
import Fectoria.TvAbstractFactory;

public class EnsamblajeTV {
    public EnsamblajeTV(TvAbstractFactory factory){
        Color color=factory.createColor();
        Tv tv = factory.createTV();
        color.colorea(tv);
    }
}
```

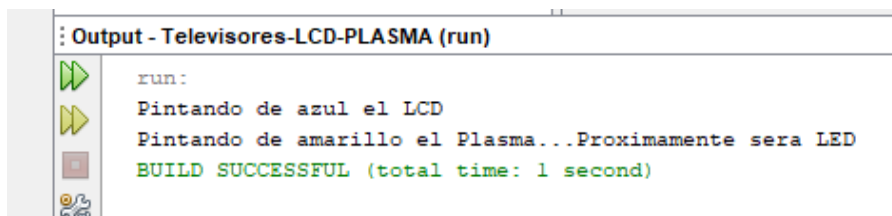
Y, por último, el main:

Clase Cliente (main)

```
package Cliente;
import Factoria.FactoryLcdAzul;
import Factoria.FactoryPlasmaAmarillo;
import Factoria.TvAbstractFactory;

public class Cliente {
    public static void main(String[] args) {
        //probando el factory LCD + Azul
        TvAbstractFactory f1 = new FactoryLcdAzul();
        EnsamblajeTV e = new EnsamblajeTV(f1);
        //probando el factory plasma + Amarillo
        TvAbstractFactory f2 = new FactoryPlasmaAmarillo();
        EnsamblajeTV e2 = new EnsamblajeTV(f2);
    }
}
```

ALGUNAS PRUEBAS....



BUILDER

Este patrón tiene por objetivo facilitar la instanciación de objetos de una determinada clase o familia de clases, entregando objetos cuyos atributos posean valores definidos sin recurrir a su llenado mediante constructores ni métodos de tipo setter en otras palabras Este es un patrón que es muy simple pero muy útil, el cual nos permite crear objetos complejos a través de uno más simple.

DIAGRAMA DE CLASES

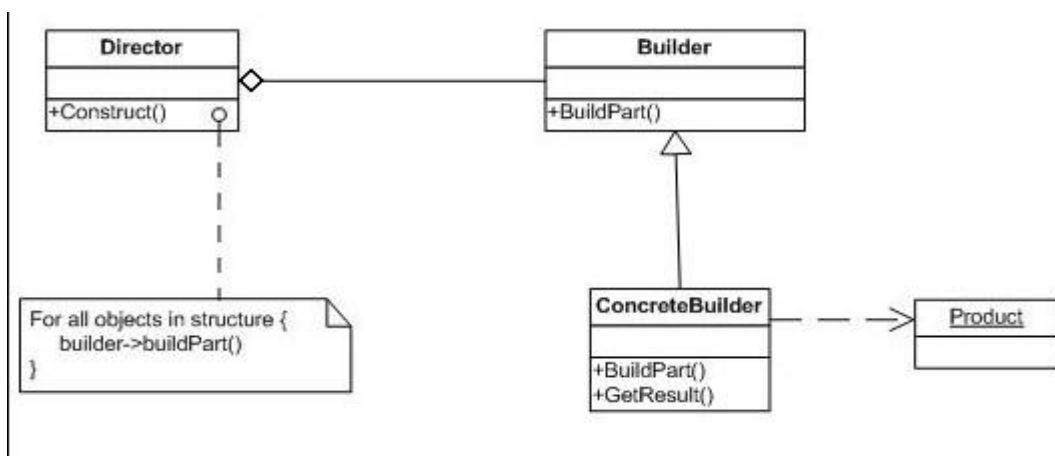
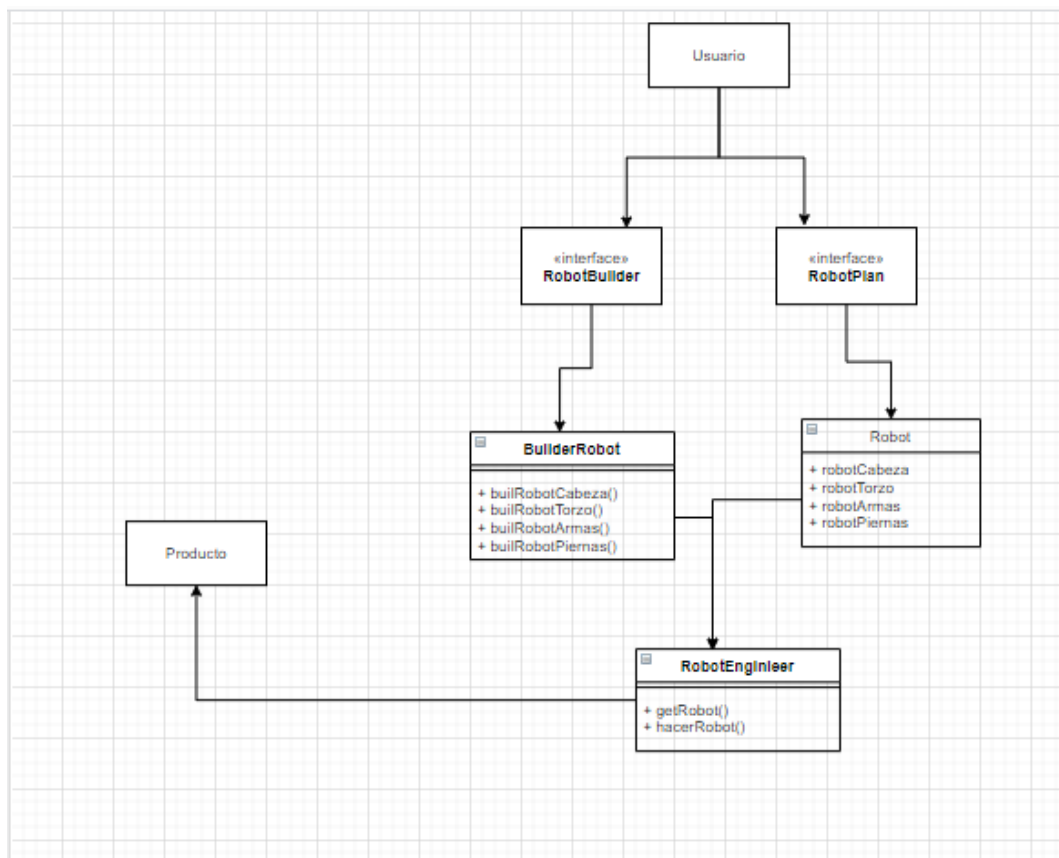


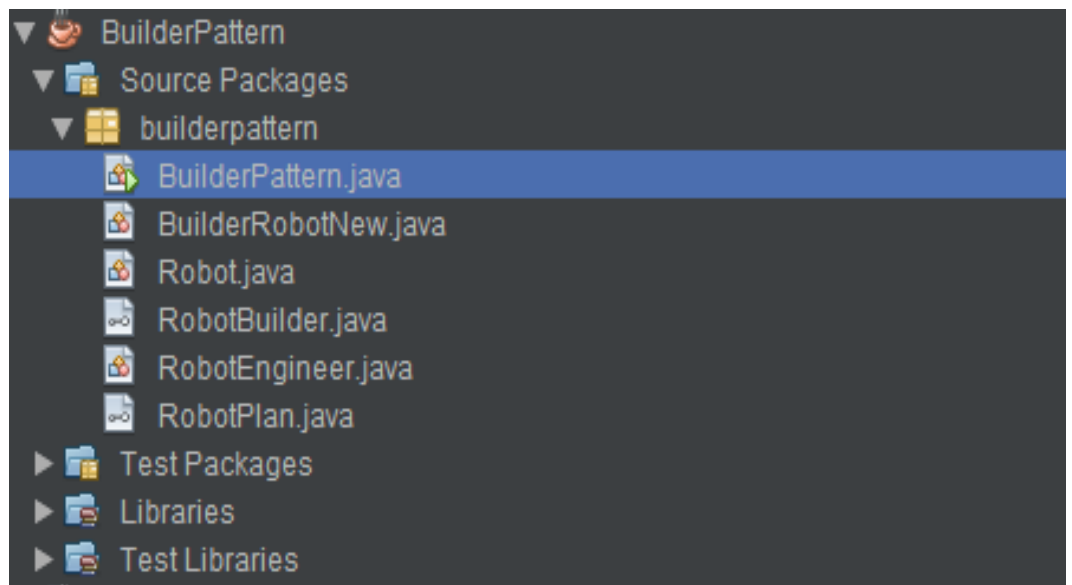
Diagrama de Clases Genérico

APLICACIÓN

Se requiere arma un robot con diferentes partes y poder ensamblar haciendo uso del patrón de diseño builder. Al implementar este patrón garantizamos que partes queremos ensamblar y en qué orden podemos ensamblar el robot



En este diagrama se representa como el usuario crea el robot, utilizando las interfaces constructor y interfaz plan que permite ordenar la construcción del robot



Estructura de Carpetas

Interface RobotPlan

```
/**
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package builderpattern;

/**
 *
 * @author LENOVO
 */
public interface RobotPlan {

    public void setRobotCabeza(String cabeza);

    public void setRobotTorso(String torso);

    public void setRobotArmas(String armas);

    public void setRobotPiernas(String piernas);

}
```

Clase RobotEngineer

```
public class RobotEngineer {  
  
    private RobotBuilder robotBuilder;  
  
    public RobotEngineer(RobotBuilder robotBuilder) {  
        this.robotBuilder = robotBuilder;  
    }  
  
    public Robot getRobot() {  
        return this.robotBuilder.getRobot();  
    }  
  
    public void hacerRobot() {  
  
        this.robotBuilder.buildRobotCabeza();  
  
        this.robotBuilder.buildRobotTorso();  
  
        this.robotBuilder.buildRobotArmas();  
  
        this.robotBuilder.buildRobotPiernas();  
  
    }  
}
```

Interface RobotBuilder

```
public interface RobotBuilder {  
  
    public void buildRobotCabeza();  
  
    public void buildRobotTorso();  
  
    public void buildRobotArmas();  
  
    public void buildRobotPiernas();  
  
    public Robot getRobot();  
}
```


Clase BuilderPattern

```
public class BuilderPattern {  
  
    /**  
     * @param args the command line arguments  
     */  
    public static void main(String[] args) {  
        RobotBuilder antiguoStiloRobot = new BuilderRobotNew();  
  
        // Pase la especificación BuilderRobotNew a la ingeniera  
        RobotEngineer robotEngineer = new RobotEngineer(antiguoStiloRobot);  
  
        // Se le dice al ingeniero que haga el robot usando las especificaciones  
        // de la clase BuilderRobotNew  
        robotEngineer.hacerRobot();  
  
        // El ingeniero devuelve el robot correcto basado en la especificación  
        Robot firstRobot = robotEngineer.getRobot();  
  
        System.out.println("Construcción Robot");  
  
        System.out.println("Tipo de cabeza de robot: " + firstRobot.getRobotCabeza());  
  
        System.out.println("Tipo de torso robot: " + firstRobot.getRobotTorso());  
  
        System.out.println("Tipo de armas robot: " + firstRobot.getRobotArmas());  
  
        System.out.println("Tipo de piernas robot: " + firstRobot.getRobotPiernas());  
    }  
}
```

Clase Robot

```
public class Robot implements RobotPlan {

    private String robotCabeza;

    private String robotTorso;

    private String robotArmas;

    private String robotPiernas;

    public void setRobotCabeza(String cabeza) {
        robotCabeza = cabeza;
    }

    public void setRobotTorso(String torso) {
        robotTorso = torso;
    }

    public void setRobotArmas(String armas) {
        robotArmas = armas;
    }

    public void setRobotPiernas(String piernas) {
        robotPiernas = piernas;
    }

    public String getRobotCabeza() {
        return robotCabeza;
    }

    public String getRobotTorso() {
        return robotTorso;
    }

    public String getRobotArmas() {
        return robotArmas;
    }

    public String getRobotPiernas() {
        return robotPiernas;
    }

}
```

Clase BuilderRobotNew

```
public class BuilderRobotNew implements RobotBuilder {

    private Robot robot;

    public BuilderRobotNew() {
        this.robot = new Robot();
    }

    @Override
    public void buildRobotCabeza() {
        this.robot.setRobotCabeza("CABEZA REDONDA");
    }

    @Override
    public void buildRobotTorso() {
        this.robot.setRobotTorso("TORZO ANCHO");
    }

    @Override
    public void buildRobotArmas() {
        this.robot.setRobotArmas("PISTOLAS Y CAÑONES");
    }

    @Override
    public void buildRobotPiernas() {
        this.robot.setRobotPiernas("PRIENAS LARGAS");
    }

    public Robot getRobot() {
        return this.robot;
    }

}
```

Clases BuilderPattern (main)

```
public class BuilderPattern {  
  
    /**  
     * @param args the command line arguments  
     */  
    public static void main(String[] args) {  
        RobotBuilder antiguoStiloRobot = new BuilderRobotNew();  
  
        // Pase la especificación BuilderRobotNew a la ingeniera  
        RobotEngineer robotEngineer = new RobotEngineer(antiguoStiloRobot);  
  
        // Se le dice al ingeniero que haga el robot usando las especificaciones  
        // de la clase BuilderRobotNew  
        robotEngineer.hacerRobot();  
  
        // El ingeniero devuelve el robot correcto basado en la especificación  
        Robot firstRobot = robotEngineer.getRobot();  
  
        System.out.println("Construcción Robot");  
  
        System.out.println("Tipo de cabeza de robot: " + firstRobot.getRobotCabeza());  
  
        System.out.println("Tipo de torso robot: " + firstRobot.getRobotTorso());  
  
        System.out.println("Tipo de armas robot: " + firstRobot.getRobotArmas());  
  
        System.out.println("Tipo de piernas robot: " + firstRobot.getRobotPiernas());  
    }  
}
```

ALGUNAS PRUEBAS....

```
run:  
Construcción Robot  
Tipo de cabeza de robot: CABEZA REDONDA  
Tipo de torso robot: TORZO ANCHO  
Tipo de armas robot: PISTOLAS Y CAÑONES  
Tipo de piernas robot: PRIENAS LARGAS  
BUILD SUCCESSFUL (total time: 0 seconds)
```

En este resultado podemos visualizar como se ha creado el robot con sus diferentes partes y los diferentes tipos de accesorios

FACTORY METHOD

Define una interfaz para crear un objeto, pero deja en manos de las subclases la decisión de qué clase concreta e instancie.

DIAGRAMA DE CLASES

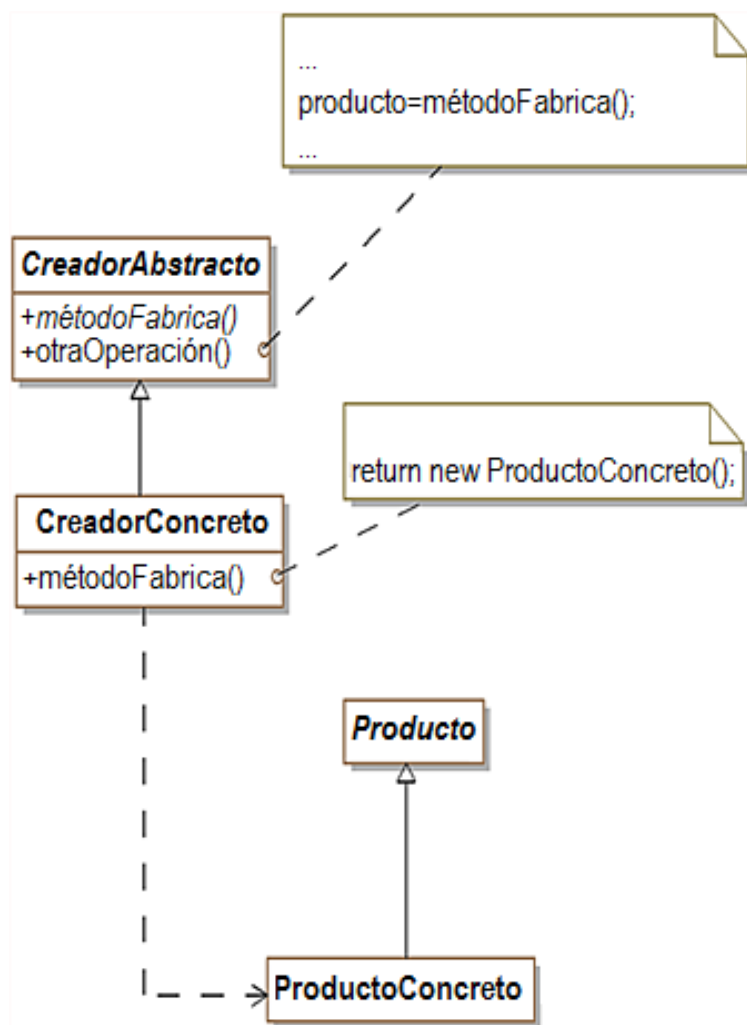
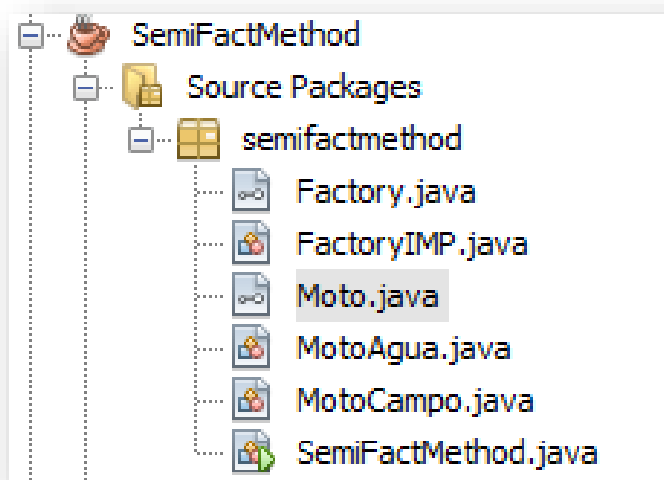
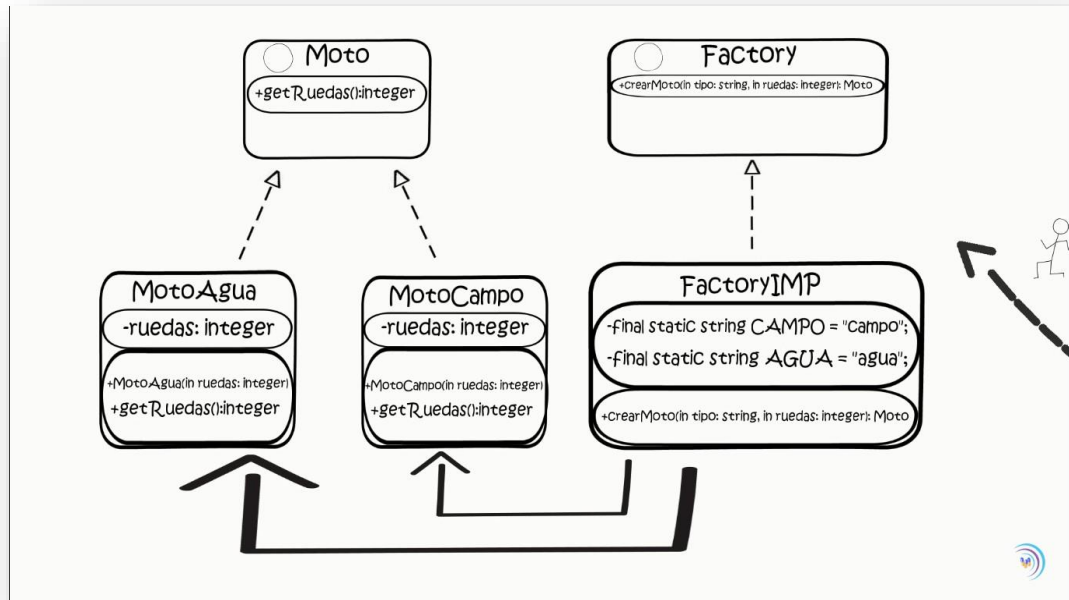


Diagrama de Clases Genérico

APLICACIÓN

Se desea ensamblar las piezas de una moto que permita su construcción para varios terrenos de acuerdo a las especificaciones del cliente.



Estructura de Carpetas

Clase MotoCampo

```
public class MotoCampo implements Moto{  
    int ruedas;  
  
    public MotoCampo(int ruedas){  
        this.ruedas = ruedas;  
    }  
  
    public int getRuedas(){  
        return this.ruedas;  
    }  
}
```

Clase MotoAgua

```
public class MotoAgua implements Moto {  
    int ruedas;  
  
    public MotoAgua(int ruedas){  
        this.ruedas = ruedas;  
    }  
  
    public int getRuedas(){  
        return this.ruedas;  
    }  
}
```

Interface Moto

```
public interface Moto {  
    public int getRuedas();  
}
```

Interface Factory

```
public interface Factory {  
    public Moto creaMoto(String tipo, int ruedas);  
}
```

Clase FactoryIMP

```
public class FactoryIMP implements Factory{  
    public final static String AGUA = "agua";  
    public final static String CAMPO = "campo";  
  
    public Moto creaMoto(String tipo, int ruedas){  
        switch(tipo){  
            case AGUA: return new MotoAgua(ruedas);  
            case CAMPO: return new MotoCampo(ruedas);  
            default: return null;  
        }  
    }  
}
```


Clase SemiFactMethod (main)

```
public class SemiFactMethod {  
  
    /**  
     * @param args the command line arguments  
     */  
    public static void main(String[] args) {  
  
        String tipoMoto = "campo";  
        int numRuedas= 4;  
        Factory mifactoria = new FactoryIMP();  
        Moto miMoto = mifactoria.creaMoto(tipoMoto, numRuedas);  
        System.out.println("Es una moto de " + miMoto.getRuedas()+ " ruedas." );  
    }  
}
```

ALGUNAS PRUEBAS....

run:

Es una moto de 4 ruedas.

BUILD SUCCESSFUL (total time: 1 second)

PROTOTYPE

Prototipo es un patrón de diseño de software creacional cuyo primordial objetivo es crear a partir de un modelo; el funcionamiento de este patrón es simple ya que su propósito consiste en realizar una o varias copias exactas de otro objeto, permitiendo la utilización de atributos ya existentes sin requerir la creación de nuevos objetos. (Gamma, Vlissides, Johnson, & Helm, 1994)

DIAGRAMA DE CLASES

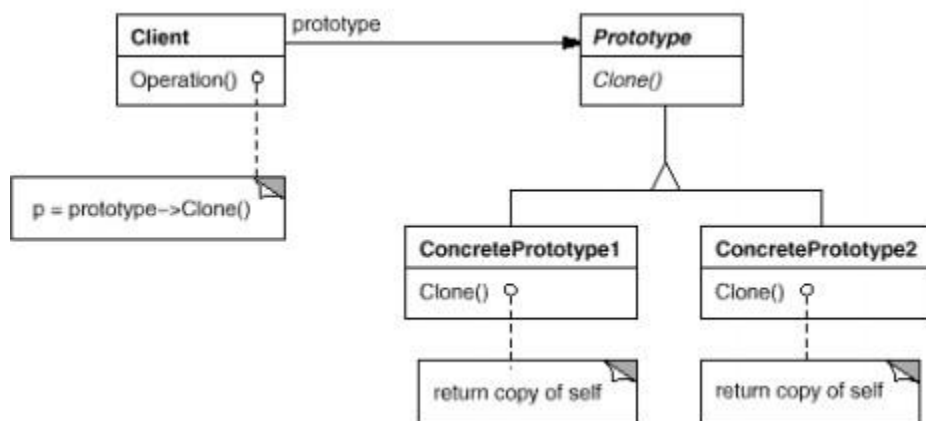
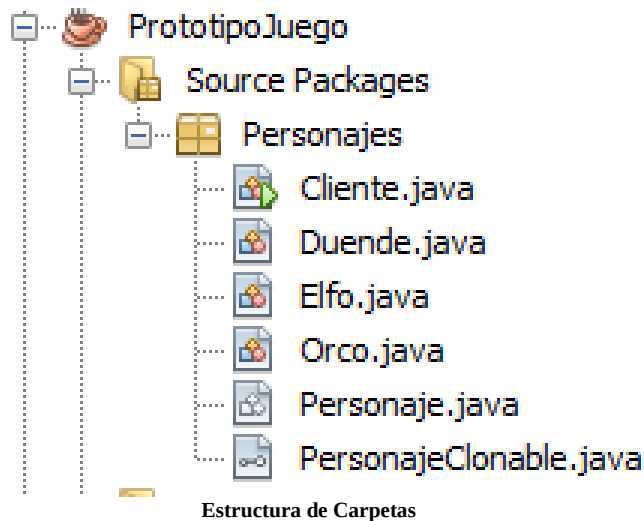
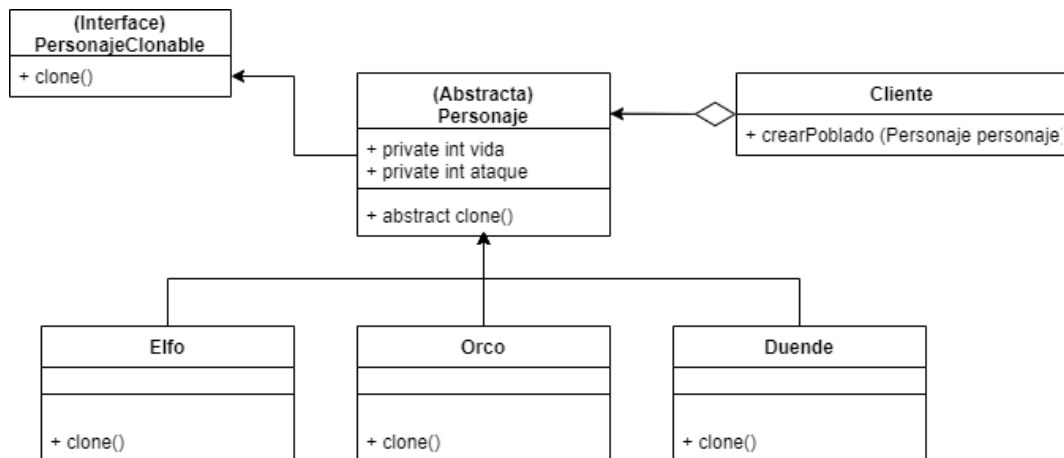


Diagrama de Clases Genérico

APLICACIÓN

Para un videojuego de roles es necesaria la creación de tres tipos de personajes (Elfos, Duendes y Orcos) donde cada rol tendrá un puntaje de vida y ataque particular.



Clase Personaje

La clase Personaje es una clase abstracta que define los atributos vida y ataque que estarán presentes en todos los tipos de rol, en esta clase implementa la clase interfaz clonable (PersonajeClonable) y la creación de todos los personajes se realizara a través de esta.

```
1 package Personajes;
2
3 /**
4  *
5  * @author agerena
6  */
7
8 public abstract class Personaje implements PersonajeClonable{
9
10     public int vida;
11     public int ataque;
12
13     public Personaje(int vida, int ataque){
14
15         this.vida = vida;
16         this.ataque = ataque;
17     }
18
19     @Override
20     public abstract PersonajeClonable clone ();
21
22 }
23
```

Clase PersonajeClonable

La clase PersonajeClonable es de tipo interfaz clonable y en ella se declara el método clon que se utilizara para la creación de los prototipos desde la clase Personajes.

```
1 package Personajes;
2
3 /**
4  *
5  * @author agerena
6  */
7
8 public interface PersonajeClonable extends Cloneable{
9
10     public abstract PersonajeClonable clone();
11
12 }
```

Clase Orco

La clase Orco cuenta con los atributos propios del tipo de rol, esta clase es extendida a la clase Personajes y mediante el método clon realiza la creación de los personajes cuyo rol es Orco, asignando los puntajes de vida y ataque definidos,

```
1 package Personajes;
2
3 /**
4  *
5  * @author agerena
6  */
7 public class Orco extends Personaje{
8
9     public static final int VIDA = 90;
10    public static final int ATAQUE = 80;
11
12    public Orco () {
13        super (VIDA, ATAQUE);
14    }
15
16    @Override
17    public PersonajeClonable clone () {
18        Orco clon = new Orco();
19        clon.vida = this.vida;
20        clon.ataque = this.ataque;
21        return clon;
22    }
23
24    @Override
25    public String toString(){
26        return "orco";
27    }
28 }
```

Clase Elfo

La clase Elfo cuenta con los atributos propios del tipo de rol, esta clase es extendida a la clase Personajes y mediante el método clon realiza la creación de los personajes cuyo rol es Elfo, asignando los puntajes de vida y ataque definidos.

```
1 package Personajes;
2
3 /**
4  *
5  * @author agerena
6  */
7 public class Elfo extends Personaje{
8
9     public static final int VIDA = 100;
10    public static final int ATAQUE = 120;
11
12    public Elfo () {
13        super(VIDA,ATAQUE);
14    }
15
16    @Override
17    public PersonajeClonable clone () {
18        Elfo clon = new Elfo();
19        clon.vida = this.vida;
20        clon.ataque = this.ataque;
21        return clon;
22    }
23
24    @Override
25    public String toString(){
26        return "elfo";
27    }
28 }
```

Clase Duende

La clase Duende cuenta con los atributos propios del tipo de rol, esta clase es extendida a la clase Personajes y mediante el método clon realiza la creación de los personajes cuyo rol es Duende, asignando los puntajes de vida y ataque definidos.

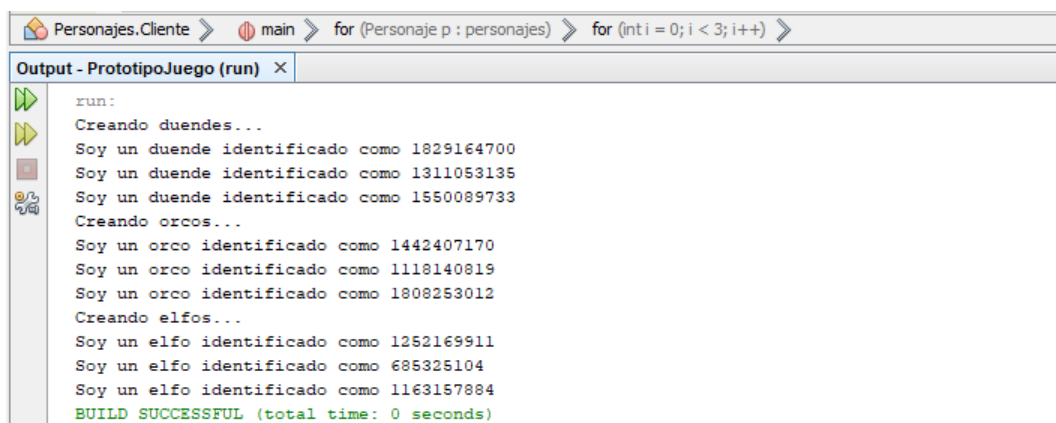
```
1 package Personajes;
2
3 /**
4  *
5  * @author agerena
6  */
7
8 public class Duende extends Personaje{
9
10     public static final int VIDA = 70;
11     public static final int ATAQUE = 50;
12
13     public Duende () {
14         super(VIDA, ATAQUE);
15     }
16
17     @Override
18     public PersonajeClonable clone () {
19         Duende clon = new Duende();
20         clon.vida = this.vida;
21         clon.ataque = this.ataque;
22         return clon;
23     }
24
25     @Override
26     public String toString(){
27         return "duende";
28     }
29 }
```

Clase Cliente (main)

A través de la clase cliente se realiza la creación del número de personajes definido por el usuario llamando el método de cada tipo de rol, para la creación de los personajes se agrega un código hash que permita identificar que aunque los personajes tienen igual nombre y puntajes cuentan con un identificador propio siendo una clon idéntico.

```
1 package Personajes;
2
3 /**
4  *
5  * @author agerena
6  */
7 public class Cliente {
8
9     public static void main(String[] args) {
10
11         Personaje[] personajes = new Personaje[3];
12         personajes[0] = new Duende();
13         personajes[1] = new Orco();
14         personajes[2] = new Elfo();
15
16         for (Personaje p : personajes) {
17             System.out.println("Creando " + p.toString() + "s...");
18             for (int i = 0; i < 3; i++) {
19                 System.out.println("Soy un " + p.clone() + " identificado como "
20                     + System.identityHashCode(System.identityHashCode(p.clone())));
21             }
22         }
23     }
24 }
25
26 }
```

ALGUNAS PRUEBAS....



```
Personajes.Cliente > main > for (Personaje p : personajes) > for (inti = 0; i < 3; i++) >
Output - PrototipoJuego (run) X
run:
Creando duendes...
Soy un duende identificado como 1829164700
Soy un duende identificado como 1311053135
Soy un duende identificado como 1550089733
Creando orcos...
Soy un orco identificado como 1442407170
Soy un orco identificado como 1118140819
Soy un orco identificado como 1808253012
Creando elfos...
Soy un elfo identificado como 1252169911
Soy un elfo identificado como 685325104
Soy un elfo identificado como 1163157884
BUILD SUCCESSFUL (total time: 0 seconds)
```


SINGLETON

El objetivo de este patrón es el de crear una única instancia de cada sistema, de esta manera evitando la duplicación de registros dentro de este, o ejecución múltiple de procedimientos del mismo, con ello logrando establecer un punto de acceso global a la clase, con el fin de evitar errores de confiabilidad y veracidad en los datos o registros.

DIAGRAMA DE CLASES

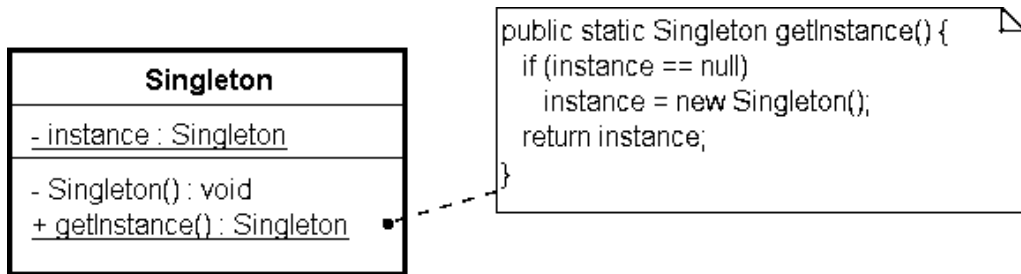
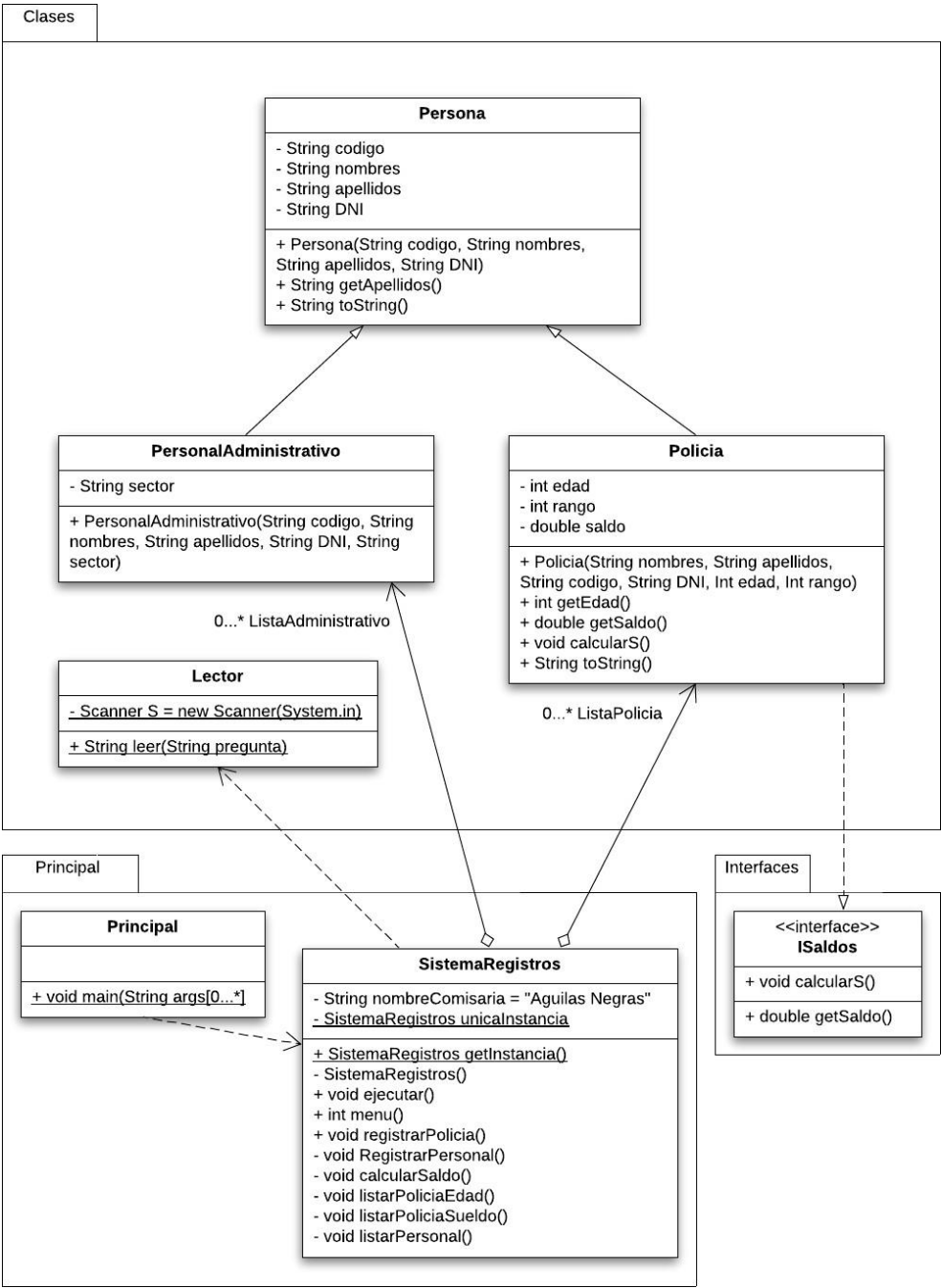
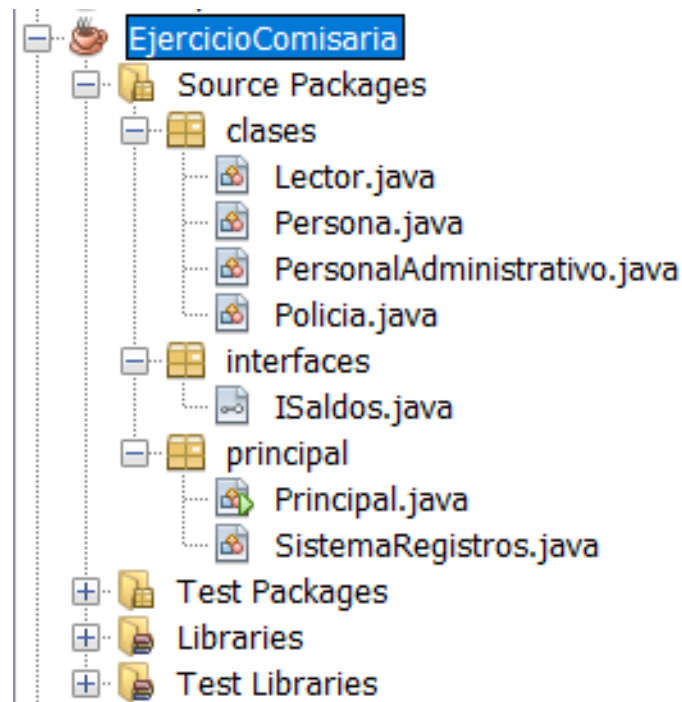


Diagrama de Clases Genérico

APLICACIÓN

Para un videojuego de roles es necesaria la creación de tres tipos de personajes (Elfos, Duendes y Orcos) donde cada rol tendrá un puntaje de vida y ataque particular.





Estructura de Carpetas

Clase Persona

```
1 package clases;
2
3 public class Persona{
4     private String codigo;
5     private String nombres;
6     private String apellidos;
7     private String DNI;
8
9     public Persona(String codigo, String nombres, String apellidos, String DNI) {
10         this.codigo = codigo;
11         this.nombres = nombres;
12         this.apellidos = apellidos;
13         this.DNI = DNI;
14     }
15
16     public String getApellidos() {
17         return apellidos;
18     }
19
20     @Override
21     public String toString() {
22         return "\nCódigo: "+codigo
23             +"\nNombres: "+nombres
24             +"\nApellidos: "+apellidos
25             +"\nDNI: "+DNI;
26     }
27 }
28
29 }
```

Clase PersonalAdministrativo

```
1 package clases;
2
3 public class PersonalAdministrativo extends Persona {
4     private String sector;
5
6     public PersonalAdministrativo(String codigo, String nombres, String apellidos, String DNI, String sector) {
7         super(codigo, nombres, apellidos, DNI);
8         this.sector = sector;
9     }
10
11 }
12
```

Clase Policia

```
1 package clases;
2
3 import interfaces.ISaldos;
4
5 public class Policia extends Persona implements ISaldos {
6     private int edad;
7     private int rango;
8     private double saldo;
9
10    public Policia(String nombres, String apellidos, String codigo, String DNI, int edad, int rango) {
11        super(codigo, nombres, apellidos, DNI);
12        this.edad=edad;
13        this.rango=rango;
14    }
15
16    public int getEdad() {
17        return edad;
18    }
19
20    @Override
21    public double getSaldo() {
22        return saldo;
23    }
24
25    @Override
26    public void calcularS() {
27        saldo = 120 * rango;
28    }
29
30    @Override
31    public String toString() {
32        return super.toString()
33            + "\nEdad: "+edad
34            + "\nRango: "+rango
35            + "\nSaldo: "+saldo;
36    }
37 }
```

Interface ISaldos

```
1 package interfaces;
2
3 public interface ISaldos {
4     public void calcularS();
5     public double getSaldo();
6 }
7
```

Clase Lector

```
1 package clases;
2
3 import java.util.Scanner;
4
5 public class Lector { // Wrapper.
6
7     private static Scanner S = new Scanner(System.in);
8
9     public static String leer(String pregunta) {
10         System.out.println(pregunta);
11         return S.nextLine();
12     }
13
14 }
15
```

Clase Principal

```
1 package principal;
2
3 public class Principal {
4
5     public static void main(String[] args) {
6         SistemaRegistros.getInstancia().ejecutar();
7         // new SistemaRegistros().ejecutar();
8         // SistemaRegistros S = new SistemaRegistros();
9     }
10
11 }
12
```

Clase SistemaRegistros

```
1 package principal;
2
3 import clases.Lector;
4 import clases.PersonalAdministrativo;
5 import clases.Policia;
6 import java.util.ArrayList;
7
8 public class SistemaRegistros {
9
10     private static SistemaRegistros unicaInstancia;
11
12     public static SistemaRegistros getInstancia() {
13         if(unicaInstancia == null)
14             unicaInstancia = new SistemaRegistros();
15         return unicaInstancia;
16     }
17
18     private SistemaRegistros() {}
19
20     private ArrayList<Policia> ListaPolicia = new ArrayList();
21     private ArrayList<PersonalAdministrativo> ListaAdministrativo = new ArrayList();
22     private final String nombreComisaria = "AGUILAS NEGRAS";
23
24     public void ejecutar() {
25         int opt;
26         do{
27             opt = menu();
28             switch (opt){
29                 case 1:
30                     registrarPolicia();
31                     break;
32                 case 2:
33                     registrarPersonal();
34                     break;
35                 case 3:
36                     calcularSaldo();
37                     break;
38                 case 4:
39                     listarPoliciaEdad();
40                     break;
41                 case 5:
42                     listarPoliciaSueldo();
43                     break;
44                 case 6:
45                     listarPersonal();
46                     break;
```

```

47         case 7:
48             System.out.println("Saliendo del programa");
49             break;
50         default:
51             System.out.println("La opcion no es valida");
52     }
53
54     }while (opt!=7);
55 }
56
57 public int menu(){
58     String opciones=
59         "\n MENU PRINCIPAL DE LA COMISARÍA "+nombreComisaria+
60         "\n1.Registrar Policia "+
61         "\n2.Registrar PersonalAdministrativo"+
62         "\n3.Calcular el saldo del policia"+
63         "\n4.Listar ascendentemente los Policias por edad"+
64         "\n5.Listar Segun el sueldo de los policias"+
65         "\n6.Listar alfabeticamente al PersonalAdministrativo por apellidos"+
66         "\n7.Salir"+
67         "\nSeleccione Opcion: ";
68     return Integer.parseInt(Lector.leer(opciones));
69 }
70
71
72 public void registrarPolicia(){
73     String nombre=Lector.leer("Ingrese nombre: ");
74     String apellido=Lector.leer("Ingrese apellidos: ");
75     String codigo=Lector.leer("Ingrese codigo: ");
76     String DNI=Lector.leer("Ingrese DNI: ");
77     int edad=Integer.parseInt(Lector.leer("Ingrese su edad: "));
78     int rango = -1;
79     while(rango<1||rango>5) // while( !(rango>=1&& rango<=5) )
80     {
81         rango = Integer.parseInt(Lector.leer("Ingrese su rango: "));
82         do{
83             rango = Integer.parseInt(Lector.leer("Ingrese su rango: "));
84         }while(rango<1||rango>5);
85     }
86     ListaPolicia.add( new Policia(nombre,apellido,codigo,DNI,edad,rango) );

```

```

87 -         private void registrarPersonal() {
88             3cxing codigo = Leczor.deer("Ing-es= --diJz: ");
89
90             String apelliaos = Lector.leer("Ir.g.eve a&eiil::s. "1:
91             3txing DNI = Lectox.deer("Ing-eve ->1.I: ");
92
93             LiscaAdmin*scraziw. add( new PersonalAdministrativo(codigo, nombres, apsliaos, DNI,
94             sector) );
95
96 |-         private void calcularsalao() {
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133 -         for(Poicicia p : LiszaPoiacia)
134             p.calcularS();
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```


Luego de la correcta implementación del patrón no podemos realizar lo siguiente:

Clase Principal (main)

```
1 package principal;
2
3 public class Principal {
4
5     public static void main(String[] args) {
6         SistemaRegistros.getInstance().ejecutar();
7         new SistemaRegistros().ejecutar();
8         // SistemaRegistros S = new SistemaRegistros();
9     }
10
11 }
12
```

No podemos instanciar la clase ya que tenemos la restricción con el constructor de la clase al ponerlo privado y también al haber establecido el método de la clase de tipo privado y estático como se mostrará a continuación:

Clase SistemaRegistros

```
8 public class SistemaRegistros {
9
10     private static SistemaRegistros unicaInstancia;
11
12     public static SistemaRegistros getInstance() {
13         if(unicaInstancia == null)
14             unicaInstancia = new SistemaRegistros();
15         return unicaInstancia;
16     }
17
18     private SistemaRegistros() {}
19 }
```

Como se muestra en la siguiente representación tampoco es posible:

Clase Principal (main)

```
1 package principal;
2
3 public class Principal {
4
5     public static void main(String[] args) {
6         SistemaRegistros.getInstance().ejecutar();
7         // new SistemaRegistros().ejecutar();
8         SistemaRegistros S = new SistemaRegistros();
9     }
10
11 }
12
```

ALGUNAS PRUEBAS....

```
MENU PRINCIPAL DE LA COMISARÍA AGUILAS NEGRAS
1.Registrar Policia
2.Registrar PersonalAdministrativo
3.Calcular el saldo del policia
4.Listar ascendentemente los Policias por edad
5.Listar Segun el sueldo de los policias
6.Listar alfabeticamente al PersonalAdministrativo por apellidos
7.Salir
Seleccione Opcion:
```

Podemos registrar información del personal del sistema, calcular los sueldos según su rango, listar los policías o personal administrativo, para motivos de ejemplo listaremos el personal policial por orden ascendente según la edad.

MENU PINCIPAL DE LA COMISARÍA AGUILAS NEGRAS

- 1.Registrar Policia
- 2.Registrar PersonalAdministrativo
- 3.Calcular el saldo del policia
- 4.Listar ascendentemente los Policias por edad
- 5.Listar Segun el sueldo de los policias
- 6.Listar alfabeticamente al PersonalAdministrativo por apellidos
- 7.Salir

Seleccione Opcion:

4

Código: 789456

Nombres: Johan

Apellidos: Meza

DNI: 159852

Edad: 22

Rango: 2

Saldo: 0.0

Código: 1098785205

Nombres: Jhon Mario

Apellidos: Meza Rios

DNI: 159753

Edad: 24

Rango: 1

Saldo: 120.0